

Practica de Vulnerabilidades Android

Evidencias del simulador Android Security Lab

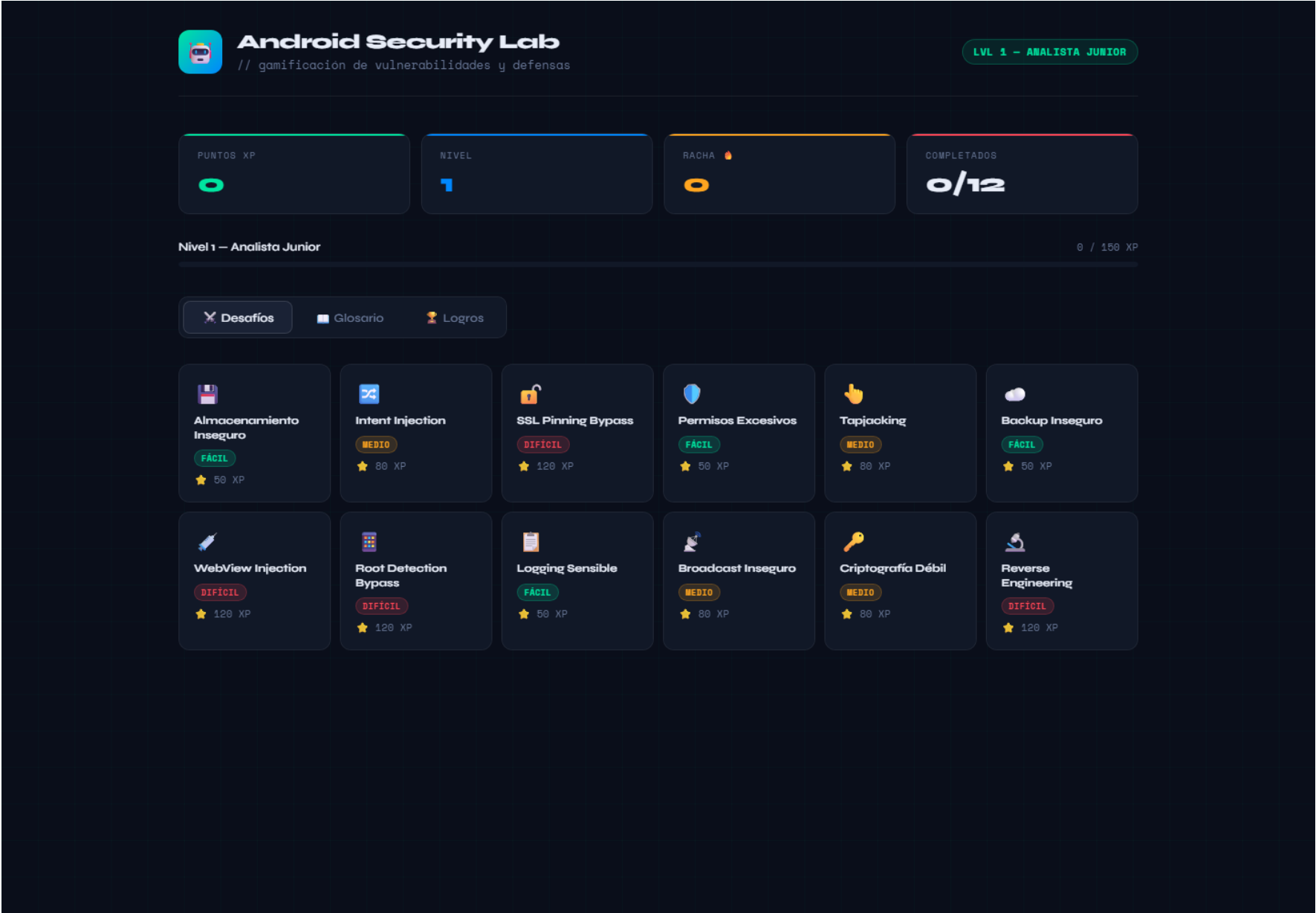
Carné	221087
Estudiante	Gabriel Alberto Paz González
Fecha	1 de junio de 2026
Archivo base	android_security_game.html

Resumen

Desafíos completados	12/12
Respuestas correctas	12/12
Puntaje final	1000 XP
Rango obtenido	Red Teamer
Logros desbloqueados	8/8

Capturas de evidencia

Inicio del laboratorio



Seleccion de desafio

Se selecciono Almacenamiento Inseguro

Almacenamiento InseguroFÁCIL★ 50 XP

Intent InjectionMEDIO★ 80 XP

OOB Pinning BypassDIFÍCIL★ 120 XP

Permisos ExcesivosFÁCIL★ 50 XP

ZiplockingMEDIO★ 80 XP

Backup InseguroFÁCIL★ 50 XP

WebView InjectionDIFÍCIL★ 120 XP

Root Detection BypassDIFÍCIL★ 120 XP

Logging SensibleFÁCIL★ 50 XP

Broadcast InseguroMEDIO★ 80 XP

Criptografía DébilMEDIO★ 80 XP

Reverse EngineeringDIFÍCIL★ 120 XP

0/12 completados

Almacenamiento InseguroFÁCIL

Las apps Android pueden guardar datos en lugares accesibles sin protección.

▲ ESCENARIO DE ATAQUE

Una app bancaria guarda el PIN en texto plano en SharedPreferences. Con ADB shell se ejecuta: `cat /data/data/com.bank.app/shared_prefs/config.xml` y el PIN aparece en texto claro.

¿Cuál es la defensa más efectiva para este escenario?

A Guardar el PIN en una variable global en memoria RAM

B Usar Android Keystore System para cifrar datos sensibles con claves ligadas al hardware

C Cambiar el nombre del archivo XML para dificultar su localización

D Eliminar los logs de la app

✳️ HERRAMIENTAS RELEVANTES

Android Keystore API

Jetpack Security

drozer

MobSF

← Volver

Siguiente →

Quiz correcto: SSL Pinning Bypass

**WebView Injection**
DIFICIL
★ 120 XP

**Root Detection Bypass**
DIFICIL
★ 120 XP

**Logging Sensible**
FÁCIL
★ 50 XP

**Broadcast Inseguro**
MEDIO
★ 80 XP

**Criptografía Débil**
MEDIO
★ 80 XP

**Reverse Engineering**
DIFICIL
★ 120 XP

2/12 completados

**SSL Pinning Bypass** DIFICIL

Interceptar tráfico de apps que no implementan Certificate Pinning.

▲ ESCENARIO DE ATAQUE

Se instala el certificado CA de Burp Suite en el dispositivo. La app no implementa pinning. Todo el tráfico HTTPS queda visible: tokens de sesión, credenciales y datos personales quedan expuestos.

¿Cómo implementar SSL/Certificate Pinning correctamente en Android?

A Usar solo HTTP en redes internas de confianza

B Bloquear todas las conexiones salientes excepto al propio servidor

C Implementar Network Security Config con pin-set + validar hash SHA-256 en OkHttp CertificatePinner con pin de respaldo

D Usar certificados autofirmados en el servidor

✅ ¡Correcto! +120 XP

Network Security Config declara pins en XML. OkHttp CertificatePinner verifica el hash SHA-256. Incluir al menos 2 pines (backup pin) para evitar lockout en rotación de certificados. TrustKit-Android simplifica la implementación y ofrece reporting.

✳️ HERRAMIENTAS RELEVANTES

Burp Suite

Frida

objection

TrustKit-Android

OkHttp CertificatePinner

← Volver

Siguiente →

🏆 Logro desbloqueado: Racha x3

Quiz correcto: Permisos Excesivos

**WebView Injection**
DIFÍCIL
★ 120 XP

**Root Detection Bypass**
DIFÍCIL
★ 120 XP

**Logging Sensible**
FÁCIL
★ 50 XP

**Broadcast Inseguro**
MEDIO
★ 80 XP

**Criptografía Débil**
MEDIO
★ 80 XP

**Reverse Engineering**
DIFÍCIL
★ 120 XP

3/12 completados

**Permisos Excesivos** FÁCIL

Apps que piden más permisos de los necesarios amplían la superficie de ataque.

▲ ESCENARIO DE ATAQUE

Una app de linterna solicita: CAMERA, READ_CONTACTS, ACCESS_FINE_LOCATION, READ_SMS, RECORD_AUDIO. Solo necesita CAMERA. Análisis estático revela que los extras envían datos a un servidor C2.

¿Cuál es la práctica correcta de gestión de permisos?

A Solicitar todos los permisos al inicio para no molestar después

B Principio de mínimo privilegio: solo permisos necesarios, just-in-time, verificados en runtime

C Declarar permisos en el Manifest pero no pedirlos al usuario

D Usar permisos de sistema para evitar el diálogo de solicitud

✅ ¡Correcto! +50 XP

Android 6.0+ requiere solicitar permisos dangerous en runtime. Solicitar contextualizados justo antes de usarlos con rationale. Usar `ActivityResultContracts.RequestPermission`. Auditar con: `aapt dump permissions app.apk`

✳️ HERRAMIENTAS RELEVANTES

MobSF

apktool

aapt

Android Studio Lint

← Volver

Siguiente →

🏆 Logro desbloqueado: Racha x3

Quiz correcto: Tapjacking

**WebView Injection**
DIFÍCIL
★ 120 XP

**Root Detection Bypass**
DIFÍCIL
★ 120 XP

**Logging Sensible**
FÁCIL
★ 50 XP

**Broadcast Inseguro**
MEDIO
★ 80 XP

**Criptografía Débil**
MEDIO
★ 80 XP

**Reverse Engineering**
DIFÍCIL
★ 120 XP

●●●●●●●●●●●●●●●●

4/12 completados

**Tapjacking** MEDIO

Overlay malicioso que intercepta toques destinados a la app víctima.

▲ ESCENARIO DE ATAQUE

Una app maliciosa dibuja una ventana TYPE_APPLICATION_OVERLAY sobre la app de pagos. El usuario cree tocar "Confirmar \$10" pero realmente confirma una transferencia de \$1,000.

¿Qué mecanismo de Android previene el Tapjacking?

A Usar un fondo de pantalla animado como distracción

B Aplicar filterTouchesWhenObscured="true" en Views críticas y verificar FLAG_WINDOW_IS_OBSCURED en el MotionEvent

C Pedir al usuario cerrar todas las apps antes de pagar

D Usar solo botones físicos del dispositivo

✅ ¡Correcto! +80 XP

filterTouchesWhenObscured="true" hace que Android descarte toques cuando la ventana está cubierta. Para lógica custom: verificar (event.getFlags() & MotionEvent.FLAG_WINDOW_IS_OBSCURED) != 0. Android 12 ya restringe tipos de overlay permitidos.

✳️ HERRAMIENTAS RELEVANTES

drozer

ADB

Tapjacking PoC tools

← Volver

Siguiente →

Quiz correcto: Backup Inseguro

WebView Injection
DIFÍCIL
★ 120 XP

Root Detection Bypass
DIFÍCIL
★ 120 XP

Logging Sensible
FÁCIL
★ 50 XP

Broadcast Inseguro
MEDIO
★ 80 XP

Criptografía Débil
MEDIO
★ 80 XP

Reverse Engineering
DIFÍCIL
★ 120 XP

5/12 completados

Backup Inseguro

FÁCIL

El mecanismo de backup puede exponer datos de apps sensibles.

▲ ESCENARIO DE ATAQUE

Con android:allowBackup="true" (default), el atacante ejecuta: `adb backup -f backup.ab com.bank.app`. Extrae la BD SQLite con historial de transacciones, tokens y preferencias.

¿Cómo proteger los datos ante backups no autorizados?

A Cifrar manualmente solo la base de datos SQLite

B Usar android:allowBackup="false" para apps sensibles, o configurar fullBackupContent con reglas de exclusión y dataExtractionRules en Android 12+

C No usar SQLite, solo guardar datos en el servidor

D Habilitar backup para detectar si alguien extrae los datos

✅ ¡Correcto! +50 XP

Para apps financieras, allowBackup="false" es lo más seguro. Si se necesita backup parcial, usar fullBackupContent para excluir directorios sensibles. En Android 12+, dataExtractionRules es el mecanismo nuevo. SQLCipher cifra la BD si se requiere backup.

🔖 HERRAMIENTAS RELEVANTES

ADB

Android Backup Extractor

MobSF

SQLCipher

← Volver

Siguiente →

Quiz correcto: WebView Injection

**WebView Injection**
DIFÍCIL
★ 120 XP

**Root Detection Bypass**
DIFÍCIL
★ 120 XP

**Logging Sensible**
FÁCIL
★ 50 XP

**Broadcast Inseguro**
MEDIO
★ 80 XP

**Criptografía Débil**
MEDIO
★ 80 XP

**Reverse Engineering**
DIFÍCIL
★ 120 XP

6/12 completados

**WebView Injection** DIFÍCIL

WebViews mal configuradas permiten XSS y acceso a APIs nativas de Android.

▲ ESCENARIO DE ATAQUE

App carga URLs externas con `setJavaScriptEnabled(true)` y `addJavascriptInterface`. Un sitio malicioso ejecuta `Android.sendSMS("hacker", "Msg")` directamente en JavaScript.

¿Cuál es el conjunto correcto de defensas para WebView?

A Usar solo HTTP en WebViews para evitar problemas con certificados

B Deshabilitar JS si no es necesario; si se usa `addJavascriptInterface`, anotar métodos con `@JavascriptInterface`, `allowlist` de URLs y `setSafeBrowsingEnabled(true)`

C Cargar solo páginas propias con URLs hardcodeadas en el APK

D Reemplazar completamente WebView por Chrome CustomTabs siempre

✅ ¡Correcto! +120 XP

@JavascriptInterface (API 17+) restringe qué métodos son visibles a JS. Validar URL con `allowlist` antes de cargar. `setAllowFileAccess(false)`, `setAllowUniversalAccessFromFileURLs(false)`. Para contenido externo, Chrome Custom Tabs es más seguro que WebView propio.

✳️ HERRAMIENTAS RELEVANTES

Burp Suite

MobSF

jadx

drozer

Android WebView Analyzer

← Volver

Siguiente →

Quiz correcto: Root Detection Bypass



WebView Injection

DIFÍCIL

★ 120 XP



Root Detection Bypass

DIFÍCIL

★ 120 XP



Logging Sensible

FÁCIL

★ 50 XP



Broadcast Inseguro

MEDIO

★ 80 XP



Criptografía Débil

MEDIO

★ 80 XP



Reverse Engineering

DIFÍCIL

★ 120 XP

7/12 completados



Root Detection Bypass

DIFÍCIL

Sin detección de root, el atacante puede inyectar código y manipular la app.

▲ ESCENARIO DE ATAQUE

El atacante usa Magisk para bypassar la detección básica. Con Frida hookea la función de autenticación para que retorne siempre true. Acceso total sin credenciales.

¿Cuál es la estrategia de defensa multicapa más efectiva?

A Solo verificar si existe el archivo /system/app/Superuser.apk

B Verificación multicapa: Play Integrity API + detección de binarios su + verificación BUILD.TAGS + integridad del runtime con hash verification

C Forzar al usuario a desinstalar Magisk antes de usar la app

D Implementar root detection solo en la versión de producción

✓ ¡Correcto! +120 XP

Play Integrity API verifica en servidores de Google la integridad del dispositivo y el APK. Combinar con detección local: binarios su, write permissions en /system, Magisk Manager. RootBeer ofrece múltiples checks. Anti-Frida: verificar puertos 27042. El objetivo es elevar el costo del ataque.

✖ HERRAMIENTAS RELEVANTES

Play Integrity API

RootBeer

Frida

Magisk

RASP solutions

← Volver

Siguiente →

Quiz correcto: Logging Sensible

**WebView Injection**
DIFÍCIL
★ 120 XP

**Root Detection Bypass**
DIFÍCIL
★ 120 XP

**Logging Sensible**
FÁCIL
★ 50 XP

**Broadcast Inseguro**
MEDIO
★ 80 XP

**Criptografía Débil**
MEDIO
★ 80 XP

**Reverse Engineering**
DIFÍCIL
★ 120 XP

●●●●●●●●●●●●●● 8/12 completados

**Logging Sensible** FÁCIL

Datos sensibles en logs de Android son accesibles para otras apps o atacantes.

▲ ESCENARIO DE ATAQUE

El desarrollador usa `Log.d("AUTH", "Token: "+userToken)` en producción. Cualquier app con READ_LOGS o acceso ADB captura tokens, contraseñas y PINs en texto claro.

¿Cuál es la práctica correcta de logging para builds de producción?

A Cambiar Log.d a Log.v para que sea menos visible

B Deshabilitar logs con BuildConfig.DEBUG, usar ProGuard para eliminarlos en release, y nunca loggear tokens, PINs ni contraseñas

C Guardar los logs en un archivo cifrado en la tarjeta SD

D Usar un tag de log genérico para dificultar la búsqueda

✅ ¡Correcto! +50 XP

ProGuard/R8 elimina llamadas Log.* en builds release. Usar if(BuildConfig.DEBUG) guards. Timber permite control granular por flavor. Desde Android 4.1+, READ_LOGS es necesario, pero el riesgo existe. Nunca loggear PII, tokens, credenciales ni datos sensibles.

✳ HERRAMIENTAS RELEVANTES

ProGuard/R8

Timber

ADB logcat

MobSF

Android Lint

← Volver

Siguiente →

🏆 Logro desbloqueado: Maestro Fácil

Quiz correcto: Criptografía Débil

**WebView Injection**
DIFÍCIL
★ 120 XP

**Root Detection Bypass**
DIFÍCIL
★ 120 XP

**Logging Sensible**
FÁCIL
★ 50 XP

**Broadcast Inseguro**
MEDIO
★ 80 XP

**Criptografía Débil**
MEDIO
★ 80 XP

**Reverse Engineering**
DIFÍCIL
★ 120 XP

10/12 completados

**Criptografía Débil** MEDIO

Algoritmos criptográficos obsoletos comprometen la confidencialidad de los datos.

▲ ESCENARIO DE ATAQUE

Análisis revela: `Cipher.getInstance("DES/ECB/NoPadding")` con clave hardcodeada "mysecret". DES tiene 56 bits (se rompe en horas), ECB no oculta patrones y la clave se extrae del APK con strings.

¿Cuál es la implementación criptográfica correcta para Android?

A Cambiar DES por 3DES y agregar Base64 encoding al resultado

B Usar AES-256-GCM con IV aleatorio por operación, claves en Android Keystore, y nunca secretos hardcodeados

C Usar SHA-1 para cifrar los datos importantes de la app

D Implementar un algoritmo de cifrado propio más complejo

✅ ¡Correcto! +80 XP

AES-256 en modo GCM provee cifrado autenticado (confidencialidad + integridad). IV único por operación es crítico en GCM. Android Keystore genera claves en TEE hardware. Google Tink simplifica implementación correcta. Prohibidos: DES, 3DES, RC4, MD5 para crypto, modo ECB.

✳️ HERRAMIENTAS RELEVANTES

Google Tink

Android Keystore

jadx

apktool

MobSF

← Volver

Siguiente →

Quiz correcto: Reverse Engineering

**WebView Injection**
DIFÍCIL
★ 120 XP

**Root Detection Bypass**
DIFÍCIL
★ 120 XP

**Logging Sensible**
FÁCIL
★ 50 XP

**Broadcast Inseguro**
MEDIO
★ 80 XP

**Criptografía Débil**
MEDIO
★ 80 XP

**Reverse Engineering**
DIFÍCIL
★ 120 XP

11/12 completados

**Reverse Engineering** DIFÍCIL

Los APKs pueden descompilarse para extraer lógica de negocio y secretos.

▲ ESCENARIO DE ATAQUE

Con JADX sobre el APK de la Play Store se encuentran: claves de API de Google Maps hardcodeadas, lógica de descuentos en Java legible y endpoints internos de administración expuestos.

¿Qué técnicas anti-reversing son más efectivas en combinación?

A Renombrar variables y métodos manualmente con nombres largos y confusos

B ProGuard/R8 para ofuscación + lógica crítica en NDK nativo + secretos en servidor/CredentialManager + verificar firma del APK en runtime

C Publicar la app solo en tiendas alternativas para dificultar el acceso

D Cifrar el APK con contraseña ZIP antes de publicar

✅ ¡Correcto! +120 XP

ProGuard/R8 ofusca y elimina código muerto. Lógica crítica en C/C++ via NDK es más difícil de reversear. NUNCA hardcodear secretos: usar APIs en servidor, Keystore, o Secret Manager. Verificar firma con `PackageManager.getPackageInfo()`. DexGuard (comercial) da protección avanzada.

✳️ HERRAMIENTAS RELEVANTES

jadx

apktool

Frida

ProGuard/R8

DexGuard

MobSF

apk1d

← Volver

🏆 Logro desbloqueado: Legenda

Herramientas y defensas revisadas



Android Security Lab

// gamificación de vulnerabilidades y defensas

LVL 4 - RED TEAMER

PUNTOS XP

1000

NIVEL

4

RACHA

12

COMPLETADOS

12/12

Nivel 4 — Red Teamer

250 / 450 XP

Desafíos

Glosario

Logros



ADB (Android Debug Bridge)

Herramienta CLI para comunicarse con un dispositivo Android. Usada por atacantes para extraer datos, instalar apps maliciosas o ejecutar comandos con privilegios.

adb shell

adb backup

adb logcat



Drozer

Framework de seguridad para Android que enumera y explota componentes exportados (Activities, Services, BroadcastReceiver, ContentProviders). Esencial en pentesting móvil.

drozer



MobSF (Mobile Security Framework)

Plataforma de análisis estático y dinámico para APKs Android e IPAs iOS. Automatiza detección de vulnerabilidades, permisos y comunicaciones de red.

MobSF



Frida

Framework de instrumentación dinámica que hooka funciones en runtime, modifica comportamiento, bypassa controles y extrae datos en tiempo de ejecución. Usado en bypass de root detection, SSL pinning, etc.

Frida

objection



JADX

Decompilador Dalvik/Java para Android. Convierte archivos DEX y APK en código Java legible. Fundamental para análisis estático de código fuente decompilado.

jadx

jadx-gui



Burp Suite

Proxy de intercepción HTTP/HTTPS líder en la industria. Captura y modifica tráfico. Usado para análisis de APIs y traffic tampering.

Burp Suite

OWASP ZAP

Logro desbloqueado: Leyenda



Android Security Lab
// gamificación de vulnerabilidades y defensas

LVL 4 - RED TEAMER

PUNTOS XP
1000

NIVEL
4

RACHA
12

COMPLETADOS
12/12

Nivel 4 — Red Teamer

250 / 450 XP

Desafíos

Glosario

Logros



Primer Paso
Completa tu primer desafío



Racha x3
3 aciertos consecutivos



Maestro Fácil
Completa los 4 desafíos fáciles



Guerrero Medio
Completa los 4 desafíos medios



Elite
Completa los 4 desafíos difíciles



Experto Android
Completa los 12 desafíos



Velocista
Llega al nivel 3



Leyenda
Alcanza 1000 XP